LABORATORIO 08

Pruebas automatizadas y CRUD de categorías

Estudiante: Jesus Castillo Sumire

Curso: Desarrollo de Aplicaciones Web

Proyecto: lab08-tests-categorias

Fecha: 19 de octubre de 2026

Objetivo

Verificar con pruebas automatizadas los servicios y endpoints de productos y categorías, incluyendo escenarios correctos, datos no válidos, duplicados y recursos inexistentes. El laboratorio también amplía la API con el mantenimiento completo de categorías.

Cómo se trabajó

Se tomó como base el proyecto de productos de la Semana 05 y se trabajó en una copia independiente. Se agregaron pruebas unitarias para las reglas de servicio, pruebas de integración HTTP con MockMvc y una API CRUD de categorías organizada en capas. Las pruebas usan H2 en memoria para mantener aislados los datos de MySQL.

Herramientas y entorno

Java 17, Spring Boot 3.5.6, Spring Web, Spring Data JPA, Bean Validation, Maven, JUnit 5, MockMvc, H2, MySQL/MariaDB de XAMPP, Visual Studio Code y Postman. Para las pruebas automatizadas no es necesario iniciar XAMPP: H2 crea una base temporal.

Flujo de la aplicación

Cliente HTTP o MockMvc → Controller → DTO y validación → Service → Repository → base de datos. Durante la ejecución normal se conecta a MySQL; durante las pruebas de integración usa H2.

Requerimientos atendidos

1. Pruebas unitarias del servicio de productos, incluyendo actualización y consulta de identificadores inexistentes.
2. Pruebas HTTP para operaciones de productos y sus respuestas.
3. CRUD de categorías con Entity, Repository, Service, DTO y Controller.
4. Validaciones, protección contra duplicados y respuestas globales 400, 404 y 409.
5. Pruebas unitarias y de integración aisladas mediante H2.

Base de datos y ejecución

La ejecución local utiliza la base laboratorio_productos. Iniciar MySQL en XAMPP y ejecutar `./mvnw.cmd spring-boot:run` desde la carpeta del proyecto. La API se sirve en `http://localhost:8080`. Para las pruebas se ejecuta `./mvnw.cmd clean test`; el perfil de pruebas usa H2 y no modifica los datos de XAMPP.

Endpoints principales

Productos: GET/POST `/api/productos`, GET/PUT/DELETE `/api/productos/{id}`, GET `/api/productos/buscar?nombre=...`. Categorías: GET/POST `/api/categorias`, GET/PUT/DELETE `/api/categorias/{id}`. Crear categoría responde 201 e incluye Location; borrar categoría responde 204.

Entidad Categoria.java

Representa cada categoría como entidad JPA y la asocia con la tabla categorias. El ID se genera automáticamente; el nombre es obligatorio y único en la base, y descripción admite hasta 255 caracteres. El constructor protegido permite a JPA reconstruir la entidad.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/model/Categoria.java

```
package com.tecsup.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "categorias")
public class Categoria {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false, unique = true, length = 80)
private String nombre;

@Column(length = 255)
private String descripcion;

protected Categoria() {
}

public Categoria(String nombre, String descripcion) {
this.nombre = nombre;
this.descripcion = descripcion;
}

public Long getId() {
return id;
}

public void setId(Long id) {
this.id = id;
}

public String getNombre() {
return nombre;
}

public void setNombre(String nombre) {
this.nombre = nombre;
}

public String getDescripcion() {
return descripcion;
}

public void setDescripcion(String descripcion) {
this.descripcion = descripcion;
}
}
```

Repositorio CategoriaRepository.java

Extiende JpaRepository para obtener operaciones CRUD sin implementarlas manualmente. Los dos métodos derivados verifican nombres duplicados ignorando mayúsculas, y en actualización excluyen el registro que ya tiene ese ID.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/repository/CategoriaRepository.java

```
package com.tecsup.repository;

import com.tecsup.model.Categoria;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CategoriaRepository extends JpaRepository<Categoria, Long> {

boolean existsByNombreIgnoreCase(String nombre);

boolean existsByNombreIgnoreCaseAndIdNot(String nombre, Long id);
}
```

DTO CategoriaDTO.java y validaciones

Define los datos que entran y salen de la API sin exponer directamente la entidad de persistencia. @NotBlank impide nombre vacío o compuesto por espacios; @Size limita el tamaño permitido. El controlador activa estas reglas con @Valid.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/dto/CategoriaDTO.java

```
package com.tecsup.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class CategoriaDTO {

private Long id;

@NotBlank(message = "El nombre de la categoría es obligatorio")
@Size(max = 80, message = "El nombre no puede superar 80 caracteres")
private String nombre;

@Size(max = 255, message = "La descripción no puede superar 255 caracteres")
private String descripcion;

public CategoriaDTO() {
}

public CategoriaDTO(Long id, String nombre, String descripcion) {
this.id = id;
this.nombre = nombre;
this.descripcion = descripcion;
}

public Long getId() {
return id;
}

public void setId(Long id) {
this.id = id;
}

public String getNombre() {
return nombre;
}

public void setNombre(String nombre) {
this.nombre = nombre;
}

public String getDescripcion() {
return descripcion;
}

public void setDescripcion(String descripcion) {
this.descripcion = descripcion;
}
}
```

Servicio CategoriaService.java

Contiene las reglas de negocio del CRUD. Limpia espacios, rechaza duplicados, convierte entidades a DTO, busca por ID y genera errores de dominio cuando la categoría no existe. @Transactional agrupa cada operación; las consultas usan solo lectura.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/service/CategoriaService.java

```
package com.tecsup.service;

import com.tecsup.dto.CategoriaDTO;
import com.tecsup.exception.DuplicateResourceException;
import com.tecsup.exception.ResourceNotFoundException;
import com.tecsup.model.Categoria;
import com.tecsup.repository.CategoriaRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@Transactional
public class CategoriaService {

private final CategoriaRepository repository;

public CategoriaService(CategoriaRepository repository) {
this.repository = repository;
}

@Transactional(readOnly = true)
public List<CategoriaDTO> listar() {
return repository.findAll().stream().map(this::toDto).toList();
}

@Transactional(readOnly = true)
public CategoriaDTO obtener(Long id) {
return toDto(encontrar(id));
}

public CategoriaDTO crear(CategoriaDTO dto) {
String nombre = dto.getNombre().trim();
if (repository.existsByNombreIgnoreCase(nombre)) {
throw new DuplicateResourceException("Ya existe una categoría con ese nombre.");
}
Categoria nueva = new Categoria(nombre, limpiar(dto.getDescripcion()));
return toDto(repository.save(nueva));
}

public CategoriaDTO actualizar(Long id, CategoriaDTO dto) {
Categoria existente = encontrar(id);
String nombre = dto.getNombre().trim();
if (repository.existsByNombreIgnoreCaseAndIdNot(nombre, id)) {
throw new DuplicateResourceException("Ya existe otra categoría con ese nombre.");
}
existente.setNombre(nombre);
existente.setDescripcion(limpiar(dto.getDescripcion()));
return toDto(repository.save(existente));
}

public void eliminar(Long id) {
Categoria existente = encontrar(id);
repository.delete(existente);
}

private Categoria encontrar(Long id) {
return repository.findById(id)
.orElseThrow(() -> new ResourceNotFoundException("No existe la categoría con ID " + id + "."));
}

private CategoriaDTO toDto(Categoria categoria) {
return new CategoriaDTO(categoria.getId(), categoria.getNombre(), categoria.getDescripcion());
}

private String limpiar(String valor) {
return valor == null || valor.isBlank() ? null : valor.trim();
}
}
```

Controlador CategoriaController.java

Publica las rutas REST y delega las reglas al servicio. Recibe y entrega JSON, valida las solicitudes, construye Location al crear una categoría (201) y devuelve 204 al eliminar.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/controller/CategoriaController.java

```
package com.tecsup.controller;

import com.tecsup.dto.CategoriaDTO;
import com.tecsup.service.CategoriaService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.util.UriComponentsBuilder;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/categorias")
public class CategoriaController {

private final CategoriaService service;

public CategoriaController(CategoriaService service) {
this.service = service;
}

@GetMapping
public ResponseEntity<List<CategoriaDTO>> listar() {
return ResponseEntity.ok(service.listar());
}

@GetMapping("/{id}")
public ResponseEntity<CategoriaDTO> obtener(@PathVariable Long id) {
return ResponseEntity.ok(service.obtener(id));
}

@PostMapping
public ResponseEntity<CategoriaDTO> crear(@Valid @RequestBody CategoriaDTO dto,
UriComponentsBuilder uriBuilder) {
CategoriaDTO creada = service.crear(dto);
URI location = uriBuilder.path("/api/categorias/{id}").buildAndExpand(creada.getId()).toUri();
return ResponseEntity.created(location).body(creada);
}

@PutMapping("/{id}")
public ResponseEntity<CategoriaDTO> actualizar(@PathVariable Long id,
@Valid @RequestBody CategoriaDTO dto) {
return ResponseEntity.ok(service.actualizar(id, dto));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
service.eliminar(id);
return ResponseEntity.noContent().build();
}
}
```

Manejo global GlobalExceptionHandler.java

Centraliza errores para que los endpoints respondan de forma coherente. Los datos inválidos devuelven 400 con errores por campo; un ID inexistente devuelve 404; un nombre duplicado o conflicto de integridad devuelve 409.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/exception/GlobalExceptionHandler.java

```
package com.tecsup.exception;

import org.springframework.http.ResponseEntity;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>> manejarErrores(MethodArgumentNotValidException ex) {
Map<String, String> errores = new HashMap<>();
ex.getBindingResult().getFieldErrors().forEach(error -> {
errores.put(error.getField(), error.getDefaultMessage());
});
return ResponseEntity.badRequest().body(errores);
}

@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<Map<String, String>> manejarNoEncontrado(ResourceNotFoundException ex) {
return ResponseEntity.status(404).body(Map.of("mensaje", ex.getMessage()));
}

@ExceptionHandler({DuplicateResourceException.class, DataIntegrityViolationException.class})
public ResponseEntity<Map<String, String>> manejarConflicto(RuntimeException ex) {
String mensaje = ex instanceof DuplicateResourceException
? ex.getMessage()
: "El registro entra en conflicto con un dato existente.";
return ResponseEntity.status(409).body(Map.of("mensaje", mensaje));
}
}
```

Servicio de productos ProductoService.java

Conserva las operaciones del proyecto base y permite probar las reglas de producto. La actualización busca primero el producto, copia los campos recibidos y guarda el existente; si no existe, devuelve null, que el controlador transforma en 404.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/service/ProductoService.java

```
package com.tecsup.service;

import com.tecsup.model.Producto;
import com.tecsup.repository.ProductoRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductoService {
private final ProductoRepository repo; // Sin @Autowired

// Inyección por constructor
public ProductoService(ProductoRepository repo) {
this.repo = repo;
}

public List<Producto> listar() {
return repo.findAll();
}

public Producto guardar(Producto p) {
return repo.save(p);
}

public Producto actualizar(Long id, Producto cambios) {
return repo.findById(id).map(existente -> {
existente.setNombre(cambios.getNombre());
existente.setPrecio(cambios.getPrecio());
existente.setStock(cambios.getStock());
existente.setCategoria(cambios.getCategoria());
return repo.save(existente);
}).orElse(null);
}

public Producto obtener(Long id) {
return repo.findById(id).orElse(null);
}

public void eliminar(Long id) {
repo.deleteById(id);
}

public List<Producto> buscarPorNombre(String nombre) {
return repo.findByNombreContainingIgnoreCase(nombre);
}
}
```

Controlador de productos ProductoController.java

Expone los endpoints de productos y delega la persistencia al servicio. Convierte el DTO recibido a entidad, valida el cuerpo, y traduce resultados ausentes a respuestas 404. También implementa búsqueda por nombre y eliminación.

Código completo implementado:

Archivo fuente: src/main/java/com/tecsup/controller/ProductoController.java

```
package com.tecsup.controller;

import java.util.List;
import com.tecsup.dto.ProductoDTO;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.tecsup.model.Producto;
import com.tecsup.service.ProductoService;

@RestController
@RequestMapping("/api/productos")
public class ProductoController {
private final ProductoService service; // Sin @Autowired

// Inyección por constructor
public ProductoController(ProductoService service) {
this.service = service;
}

@GetMapping
public ResponseEntity<List<Producto>> listar() {
return ResponseEntity.ok(service.listar());
}

@PostMapping
public ResponseEntity<?> guardar(@Valid @RequestBody ProductoDTO dto) {
Producto p = new Producto();
p.setNombre(dto.getNombre());
p.setPrecio(dto.getPrecio());
p.setStock(dto.getStock());
p.setCategoria(dto.getCategoria());
Producto guardado = service.guardar(p);
return ResponseEntity.status(201).body(guardado);
}

@GetMapping("/{id}")
public ResponseEntity<?> obtener(@PathVariable Long id) {
Producto p = service.obtener(id);
if (p == null) {
return ResponseEntity.status(404).body("Producto no encontrado");
}
return ResponseEntity.ok(p);
}

@PutMapping("/{id}")
public ResponseEntity<?> actualizar(@PathVariable Long id, @Valid @RequestBody ProductoDTO dto) {
Producto cambios = new Producto(null, dto.getNombre(), dto.getPrecio(), dto.getStock(), dto.getCategoria());
Producto actualizado = service.actualizar(id, cambios);
if (actualizado == null) {
return ResponseEntity.status(404).body("Producto no existe");
}
return ResponseEntity.ok(actualizado);
}

@GetMapping("/buscar")
public ResponseEntity<List<Producto>> buscarPorNombre(@RequestParam String nombre) {
List<Producto> productos = service.buscarPorNombre(nombre);
if (productos.isEmpty()) {
return ResponseEntity.status(404).body(null);
}
return ResponseEntity.ok(productos);
}

@DeleteMapping("/{id}")
public ResponseEntity<?> eliminar(@PathVariable Long id) {
Producto p = service.obtener(id);
if (p == null) {
return ResponseEntity.status(404).body("No existe");
}
service.eliminar(id);
return ResponseEntity.ok("Eliminado correctamente");
}
}
```

Pruebas unitarias de CategoriaService

La prueba crea un repositorio falso en memoria para aislar el servicio. Los casos verifican listado, limpieza de espacios, duplicados, consulta inexistente, actualización y eliminación sin depender de MySQL.

Código completo implementado:

Archivo fuente: src/test/java/com/tecsup/service/CategoriaServiceTest.java

```
package com.tecsup.service;

import com.tecsup.dto.CategoriaDTO;
import com.tecsup.exception.DuplicateResourceException;
import com.tecsup.exception.ResourceNotFoundException;
import com.tecsup.model.Categoria;
import com.tecsup.repository.CategoriaRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.lang.reflect.Proxy;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicLong;

import static org.junit.jupiter.api.Assertions.*;

class CategoriaServiceTest {

private CategoriaRepository repository;
private CategoriaService service;

private final Map<Long, Categoria> data = new HashMap<>();
private final AtomicLong sequence = new AtomicLong(1);

@BeforeEach
void setup() {
data.clear();
sequence.set(1);
repository = (CategoriaRepository) Proxy.newProxyInstance(
CategoriaRepository.class.getClassLoader(), new Class<?>[]{CategoriaRepository.class},
(proxy, method, args) -> switch (method.getName()) {
case "findAll" -> new ArrayList<>(data.values());
case "findById" -> Optional.ofNullable(data.get(args[0]));
case "existsByNombreIgnoreCase" -> data.values().stream()
.anyMatch(item -> item.getNombre().equalsIgnoreCase((String) args[0]));
case "existsByNombreIgnoreCaseAndIdNot" -> data.values().stream()
.anyMatch(item -> !item.getId().equals(args[1])
&& item.getNombre().equalsIgnoreCase((String) args[0]));
case "save" -> guardar((Categoria) args[0]);
case "delete" -> { data.remove(((Categoria) args[0]).getId()); yield null; }
default -> throw new UnsupportedOperationException(method.getName());
});
service = new CategoriaService(repository);
}

@Test
void listarConvierteLasEntidadesADto() {
guardar(new Categoria("Accesorios", "Periféricos"));

List<CategoriaDTO> resultado = service.listar();

assertEquals(1, resultado.size());
assertEquals("Accesorios", resultado.get(0).getNombre());
}

@Test
void crearRecortaElNombreYGuardaLaCategoria() {
CategoriaDTO creada = service.crear(new CategoriaDTO(null, " Cómputo ", " Equipos "));

assertEquals("Cómputo", creada.getNombre());
assertEquals("Equipos", creada.getDescripcion());
assertEquals(1, data.size());
}

@Test
void crearRechazaUnNombreDuplicado() {
guardar(new Categoria("Accesorios", null));

assertThrows(DuplicateResourceException.class,
() -> service.crear(new CategoriaDTO(null, "Accesorios", null)));
assertEquals(1, data.size());
}

@Test
void obtenerDevuelveCategoriaExistente() {
Categoria categoria = new Categoria("Cómputo", null);
categoria.setId(1L);
data.put(1L, categoria);
```

Archivo fuente: src/test/java/com/tecsup/service/CategoriaServiceTest.java

```
assertEquals("Cómputo", service.obtener(1L).getNombre());
}

@Test
void obtenerLanza404DeDominioSiNoExiste() {
assertThrows(ResourceNotFoundException.class, () -> service.obtener(99L));
}

@Test
void actualizarGuardaLosNuevosDatos() {
Categoria existente = new Categoria("Periféricos", "Entrada y salida");
existente.setId(4L);
data.put(4L, existente);

CategoriaDTO actualizada = service.actualizar(4L, new CategoriaDTO(null, "Accesorios", "Dispositivos"));

assertEquals("Accesorios", actualizada.getNombre());
assertEquals("Dispositivos", actualizada.getDescripcion());
assertSame(existente, data.get(4L));
}

@Test
void eliminarBorraLaCategoriaEncontrada() {
Categoria categoria = new Categoria("Varios", null);
categoria.setId(3L);
data.put(3L, categoria);

service.eliminar(3L);

assertFalse(data.containsKey(3L));
}

private Categoria guardar(Categoria categoria) {
if (categoria.getId() == null) {
categoria.setId(sequence.getAndIncrement());
}
data.put(categoria.getId(), categoria);
return categoria;
}
}
```

Pruebas de integración HTTP de CategoriaController

Arranca el contexto de Spring con MockMvc y H2. Ejecuta solicitudes HTTP simuladas y comprueba estado, cabeceras y JSON para CRUD, validación, duplicados y 404. Antes de cada caso limpia las tablas de prueba.

Código completo implementado:

Archivo fuente: src/test/java/com/tecsup/controller/CategoriaControllerTest.java

```
package com.tecsup.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tecsup.dto.CategoriaDTO;
import com.tecsup.repository.AuditoriaRepository;
import com.tecsup.repository.CategoriaRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Timeout;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
@Timeout(30)
class CategoriaControllerTest {

@Autowired
private MockMvc mockMvc;

@Autowired
private ObjectMapper objectMapper;

@Autowired
private CategoriaRepository categorias;

@Autowired
private AuditoriaRepository auditorias;

@BeforeEach
void limpiarBaseDePrueba() {
auditorias.deleteAll();
categorias.deleteAll();
}

@Test
void postCreaCategoriaYDevuelve201ConLocation() throws Exception {
mockMvc.perform(post("/api/categorias").contentType(MediaType.APPLICATION_JSON)
.content(json("Cómputo", "Equipos y componentes")))
.andExpect(status().isCreated())
.andExpect(header().exists("Location"))
.andExpect(jsonPath("$.id").isNumber())
.andExpect(jsonPath("$.nombre").value("Cómputo"));
}

@Test
void getListaYConsultaCategoriaPorId() throws Exception {
var creada = categorias.save(new com.tecsup.model.Categoria("Accesorios", "Periféricos"));

mockMvc.perform(get("/api/categorias"))
.andExpect(status().isOk())
.andExpect(jsonPath("$[0].nombre").value("Accesorios"));
mockMvc.perform(get("/api/categorias/{id}", creada.getId()))
.andExpect(status().isOk())
.andExpect(jsonPath("$.descripcion").value("Periféricos"));
}

@Test
void putActualizaCategoria() throws Exception {
var creada = categorias.save(new com.tecsup.model.Categoria("Audio", "Sonido"));

mockMvc.perform(put("/api/categorias/{id}", creada.getId())
.contentType(MediaType.APPLICATION_JSON)
.content(json("Audio y video", "Sonido e imagen")))
.andExpect(status().isOk())
.andExpect(jsonPath("$.nombre").value("Audio y video"));
}

@Test
void deleteEliminaCategoriaYDevuelve204() throws Exception {
var creada = categorias.save(new com.tecsup.model.Categoria("Varios", null));

mockMvc.perform(delete("/api/categorias/{id}", creada.getId()))
.andExpect(status().isNoContent());

assertFalse(categorias.existsById(creada.getId()));
```

Archivo fuente: src/test/java/com/tecsup/controller/CategoriaControllerTest.java

```
}

@Test
void getCategoriaInexistenteDevuelve404() throws Exception {
mockMvc.perform(get("/api/categorias/999"))
.andExpect(status().isNotFound())
.andExpect(jsonPath("$.mensaje").exists());
}

@Test
void postRechazaNombreEnBlanco() throws Exception {
mockMvc.perform(post("/api/categorias").contentType(MediaType.APPLICATION_JSON)
.content(json(" ", "Descripción")))
.andExpect(status().isBadRequest())
.andExpect(jsonPath("$.nombre").exists());
}

@Test
void postRechazaNombreDuplicadoIgnorandoMayusculas() throws Exception {
categorias.save(new com.tecsup.model.Categoria("Accesorios", null));

mockMvc.perform(post("/api/categorias").contentType(MediaType.APPLICATION_JSON)
.content(json("ACCESORIOS", null)))
.andExpect(status().isConflict())
.andExpect(jsonPath("$.mensaje").exists());
}

private String json(String nombre, String descripcion) throws Exception {
return objectMapper.writeValueAsString(new CategoriaDTO(null, nombre, descripcion));
}
}
```

Pruebas unitarias de ProductoService

Comprueba directamente operaciones del servicio de productos con un repositorio simulado, incluidos los escenarios de actualización y búsqueda de ID inexistente.

Código completo implementado:

Archivo fuente: src/test/java/com/tecsup/service/ProductoServiceTest.java

```
package com.tecsup.service;

import com.tecsup.model.Producto;
import com.tecsup.repository.ProductoRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.lang.reflect.Proxy;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.atomic.AtomicLong;

import static org.junit.jupiter.api.Assertions.*;

class ProductoServiceTest {

private ProductoRepository repo;
private ProductoService service;

private final Map<Long, Producto> data = new HashMap<>();
private final AtomicLong sequence = new AtomicLong(1);

@BeforeEach
void setup() {
data.clear();
sequence.set(1);
repo = (ProductoRepository) Proxy.newProxyInstance(
ProductoRepository.class.getClassLoader(), new Class<?>[]{ProductoRepository.class},
(proxy, method, args) -> switch (method.getName()) {
case "findAll" -> new ArrayList<>(data.values());
case "findById" -> Optional.ofNullable(data.get(args[0]));
case "save" -> guardar((Producto) args[0]);
case "deleteById" -> { data.remove(args[0]); yield null; }
case "findByNombreContainingIgnoreCase" -> data.values().stream()
.filter(item -> item.getNombre().toLowerCase().contains(((String) args[0]).toLowerCase()))
.toList();
default -> throw new UnsupportedOperationException(method.getName());
});
service = new ProductoService(repo);
}

@Test
void listarDevuelveProductosDelRepositorio() {
List<Producto> productos = List.of(new Producto(1L, "Monitor", 500, 10, "Pantallas"));
productos.forEach(this::guardar);

List<Producto> resultado = service.listar();

assertEquals(1, resultado.size());
assertEquals("Monitor", resultado.get(0).getNombre());
assertEquals(1, data.size());
}

@Test
void guardarPersisteElProducto() {
Producto producto = new Producto(null, "Laptop", 3000, 5, "Cómputo");
Producto resultado = service.guardar(producto);

assertNotNull(resultado.getId());
assertSame(producto, data.get(resultado.getId()));
}

@Test
void obtenerDevuelveProductoExistente() {
Producto producto = new Producto(1L, "Teclado", 100, 20, "Accesorios");
data.put(1L, producto);

Producto resultado = service.obtener(1L);

assertSame(producto, resultado);
assertSame(producto, data.get(1L));
}

@Test
void obtenerDevuelveNullCuandoNoExiste() {
assertNull(service.obtener(99L));
}

@Test
void actualizarModificaYGuardaElProductoExistente() {
```

Archivo fuente: src/test/java/com/tecsup/service/ProductoServiceTest.java

```
Producto existente = new Producto(7L, "Mouse", 80, 3, "Accesorios");
Producto cambios = new Producto(null, "Mouse inalámbrico", 95, 8, "Accesorios");
existente.setId(7L);
data.put(7L, existente);

Producto resultado = service.actualizar(7L, cambios);

assertSame(existente, resultado);
assertEquals("Mouse inalámbrico", resultado.getNombre());
assertEquals(95, resultado.getPrecio());
assertEquals(8, resultado.getStock());
assertSame(existente, data.get(7L));
}

@Test
void actualizarDevuelveNullCuandoNoExiste() {
assertNull(service.actualizar(44L, new Producto(null, "X", 10, 1, "Varios")));
assertTrue(data.isEmpty());
}

@Test
void eliminarSolicitaAlRepositorioBorrarElId() {
service.eliminar(5L);

assertFalse(data.containsKey(5L));
}

private Producto guardar(Producto producto) {
if (producto.getId() == null) {
producto.setId(sequence.getAndIncrement());
}
data.put(producto.getId(), producto);
return producto;
}
}
```

Pruebas de integración HTTP de ProductoController

Verifica por medio de MockMvc que las rutas de productos acepten solicitudes y devuelvan respuestas HTTP adecuadas, incluyendo CRUD y condiciones de error.

Código completo implementado:

Archivo fuente: src/test/java/com/tecsup/controller/ProductoControllerTest.java

```
package com.tecsup.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.tecsup.model.Producto;
import com.tecsup.repository.AuditoriaRepository;
import com.tecsup.repository.ProductoRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Timeout;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
@Timeout(30)
class ProductoControllerTest {

@Autowired
private MockMvc mockMvc;

@Autowired
private ObjectMapper objectMapper;

@Autowired
private ProductoRepository productos;

@Autowired
private AuditoriaRepository auditorias;

@BeforeEach
void limpiarBaseDePrueba() {
auditorias.deleteAll();
productos.deleteAll();
}

@Test
void postCreaProductoYDevuelve201() throws Exception {
var dto = productoDto("Mouse", 80, 20, "Accesorios");

mockMvc.perform(post("/api/productos").contentType(MediaType.APPLICATION_JSON)
.content(objectMapper.writeValueAsString(dto)))
.andExpect(status().isCreated())
.andExpect(jsonPath("$.id").isNumber())
.andExpect(jsonPath("$.nombre").value("Mouse"));
}

@Test
void getListaProductos() throws Exception {
productos.save(new Producto(null, "Monitor", 500, 10, "Pantallas"));

mockMvc.perform(get("/api/productos"))
.andExpect(status().isOk())
.andExpect(jsonPath("$[0].nombre").value("Monitor"));
}

@Test
void postRechazaNombreVacio() throws Exception {
var dto = productoDto("", 10, 2, "Varios");

mockMvc.perform(post("/api/productos").contentType(MediaType.APPLICATION_JSON)
.content(objectMapper.writeValueAsString(dto)))
.andExpect(status().isBadRequest())
.andExpect(jsonPath("$.nombre").exists());
}

@Test
void putActualizaProductoExistente() throws Exception {
Producto existente = productos.save(new Producto(null, "Teclado", 100, 2, "Accesorios"));

mockMvc.perform(put("/api/productos/{id}", existente.getId())
.contentType(MediaType.APPLICATION_JSON)
.content(objectMapper.writeValueAsString(productoDto("Teclado mecánico", 150, 5, "Accesorios"))))
.andExpect(status().isOk())
.andExpect(jsonPath("$.nombre").value("Teclado mecánico"))
.andExpect(jsonPath("$.precio").value(150));
}
```

Archivo fuente: src/test/java/com/tecsup/controller/ProductoControllerTest.java

```
@Test
void putDevuelve404SiElProductoNoExiste() throws Exception {
mockMvc.perform(put("/api/productos/999")
.contentType(MediaType.APPLICATION_JSON)
.content(objectMapper.writeValueAsString(productoDto("Mouse", 80, 1, "Accesorios"))))
.andExpect(status().isNotFound());
}

@Test
void deleteEliminaProductoExistente() throws Exception {
Producto existente = productos.save(new Producto(null, "Mouse", 80, 20, "Accesorios"));

mockMvc.perform(delete("/api/productos/{id}", existente.getId()))
.andExpect(status().isOk())
.andExpect(content().string("Eliminado correctamente"));

org.junit.jupiter.api.Assertions.assertFalse(productos.existsById(existente.getId()));
}

@Test
void deleteDevuelve404SiElProductoNoExiste() throws Exception {
mockMvc.perform(delete("/api/productos/999"))
.andExpect(status().isNotFound());
}

private ProductoDTOTestData productoDto(String nombre, double precio, int stock, String categoria) {
return new ProductoDTOTestData(nombre, precio, stock, categoria);
}

private record ProductoDTOTestData(String nombre, double precio, int stock, String categoria) { }
}
```

Resultados, observaciones y conclusiones

Verificación automatizada

La ejecución de `./mvnw.cmd clean test` finalizó con 28 pruebas aprobadas: 0 fallidas, 0 errores y 0 omitidas. Se distribuyen en cuatro clases con siete pruebas cada una: ProductoServiceTest, ProductoControllerTest, CategoriaServiceTest y CategoriaControllerTest.

Observaciones

1. Las pruebas unitarias verifican la lógica de los servicios de forma aislada y rápida, sin depender de la disponibilidad de MySQL.

2. MockMvc permite recorrer las rutas HTTP dentro del contexto de Spring y observar los códigos y cuerpos de respuesta.

3. H2 se usa como almacenamiento temporal de pruebas, de manera que los registros no se mezclan con los datos locales de XAMPP.

4. Las categorías ya tienen entidad y operaciones CRUD propias; el campo categoría existente en Producto continúa siendo texto para conservar la compatibilidad del proyecto base.

5. La ejecución completa de Maven pasó las 28 pruebas, incluidas validaciones, duplicados y consultas de recursos inexistentes.

Conclusiones

1. Las pruebas unitarias ayudan a detectar errores de lógica antes de comprobar la API desde Postman.

2. Las pruebas de integración verifican la comunicación entre controlador, validación, servicio y persistencia.

3. Separar la gestión de categorías en capas hace que cada parte del CRUD tenga una responsabilidad clara.

4. La base H2 permite repetir las pruebas sin alterar la base laboratorio_productos de MySQL.

5. Con 28 pruebas aprobadas, el laboratorio deja una verificación repetible que se puede ejecutar con el comando de Maven incluido.